

Lab 1: Nested Loop join

```
177 -- lab1 nested loop
178 explain (analyze, buffers)
179 select c.first_name, c.last_name, r.rental_date
180 from customer c
181 join rental r on c.customer_id = r.customer_id
182 where c.customer_id = 1;
```

Data Output Messages Notifications

Showing rows: 1 to 14 Page No: 1 of 1

QUERY PLAN	
1	Nested Loop (cost=0.28..359.16 rows=32 width=21) (actual time=1.128..8.853 rows=32.00 loops=1)
2	Buffers: shared hit=1 read=152
3	-> Index Scan using customer_pkey on customer c (cost=0.28..8.29 rows=1 width=17) (actual time=0.452..0.455 rows=1.00 loops=1)
4	Index Cond: (customer_id = 1)
5	Index Searches: 1
6	Buffers: shared hit=1 read=2
7	-> Seq Scan on rental r (cost=0.00..350.55 rows=32 width=10) (actual time=0.673..8.380 rows=32.00 loops=1)
8	Filter: (customer_id = 1)
9	Rows Removed by Filter: 16012
10	Buffers: shared read=150
11	Planning:
12	Buffers: shared hit=193 read=7 dirtied=1
13	Planning Time: 3.973 ms
14	Execution Time: 8.891 ms

Trả lời:

- customer là outer table vì được đọc trước và chỉ trả về 1 khách hàng.
- rental là inner table vì PostgreSQL dùng customer_id của khách hàng đó để tìm các lần thuê tương ứng.
- PostgreSQL chọn nested loop vì outer table chỉ có 1 dòng. Dùng hash join phải tạo hash table cho cả một bảng, tốn công hơn so với tra cứu một khách hàng cụ thể.
- loops = 1 ở inner table vì outer table có đúng 1 dòng, nên PostgreSQL chỉ cần chạy tìm kiếm trong rental một lần

Lab 2: Hash join

```

184 -- lab2 hash join
185 explain (analyze, buffers)
186 select f.title, l.name as language
187 from film f
188 join language l on f.language_id = l.language_id;

```

Data Output Messages Notifications

	QUERY PLAN
	text
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=32.129..137.888 rows=1000.00 loops=1)
2	Hash Cond: (f.language_id = l.language_id)
3	Buffers: shared read=89
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=10.658..115.973 rows=1000.00 lo...
5	Buffers: shared read=88
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=21.448..21.449 rows=6.00 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared read=1
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=21.395..21.398 rows=6.00 loo...
10	Buffers: shared read=1
11	Planning:
12	Buffers: shared hit=277 read=6 dirtied=2
13	Planning Time: 231.363 ms
14	Execution Time: 138.091 ms

Trả lời:

- language thường là bảng được hash vì nó rất nhỏ. PostgreSQL tạo hash table từ bảng này, đây là build phase.
- film là bảng được quét để dò vào hash table, đây là probe phase.
- Xem dòng memory usage bên trong node hash để ghi kết quả thực tế của máy cậu.
- Khi đặt work_mem = '64kb', với bảng language nhỏ thì gần như không thay đổi: hash table vẫn vừa RAM và thường có batches: 1. Nếu bảng được hash lớn hơn bộ nhớ, PostgreSQL sẽ chia thành nhiều batch và có thể phải dùng ổ đĩa, làm chậm truy vấn.

Lab 3: Merge join

```

201 -- lab3 merge join
202 set enable_hashjoin = off;
203 set enable_nestloop = off;
204
205 explain (analyze, buffers)|
206 select f.film_id, f.title, i.inventory_id
207 from film f
208 join inventory i on f.film_id = i.film_id
209 order by f.film_id;

```

Data Output Messages Notifications

Showing rows: 1 to 16 Page No: 1 of 1

QUERY PLAN	
1	Merge Join (cost=349.64..547.91 rows=4581 width=23) (actual time=4.844..8.130 rows=4581.00 loops=1)
2	Merge Cond: (f.film_id = i.film_id)
3	Buffers: shared hit=86 read=26
4	-> Index Scan using film_pkey on film f (cost=0.28..127.33 rows=1000 width=19) (actual time=0.020..0.787 rows=1000.00 loops=1)
5	Index Searches: 1
6	Buffers: shared hit=83 read=1
7	-> Sort (cost=349.37..360.82 rows=4581 width=6) (actual time=4.811..5.339 rows=4581.00 loops=1)
8	Sort Key: i.film_id
9	Sort Method: quicksort Memory: 300kB
10	Buffers: shared hit=3 read=25
11	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.824..2.682 rows=4581.00 loops=1)
12	Buffers: shared read=25
13	Planning:
14	Buffers: shared hit=89 read=5
15	Planning Time: 6.481 ms
16	Execution Time: 10.123 ms

Trả lời:

- Ở truy vấn đầu, kế hoạch nên có merge join.
- Nếu thấy index scan, PostgreSQL đã lấy dữ liệu theo thứ tự film_id từ index, nên không cần sort; đây là cách tối ưu hơn.
- Nếu thấy sort, nghĩa là ít nhất một bảng chưa có dữ liệu đúng thứ tự cần thiết, nên PostgreSQL phải tự sắp xếp trước khi merge.
- Sau reset all và bỏ order by, PostgreSQL thường chuyển lại hash join, vì join toàn bộ bảng bằng điều kiện = và không còn cần kết quả được sắp xếp. Tuy nhiên đây không phải quy tắc tuyệt đối: PostgreSQL sẽ chọn phương án có cost thấp nhất theo dữ liệu và statistics hiện tại.